Industrijska informatika – Laboratorijske vaje

študijsko leto 2026/27

1. vaja: **Diskretno-dogodkovna simulacija**

V naši okolici se večinoma odvijajo procesi, ki jih opišemo kot zvezne sisteme. Pri njihovi analizi pa se mnogokrat osredotočamo le na pomembne trenutke, dogodke. Paradigma modeliranja, ki se uporablja v takih primerih, se imenuje diskretno-dogodkovno modeliranje. Z diskretno-dogodkovnimi modeli lahko predstavimo najrazličnejše sisteme in procese iz narave in okolice (npr. proizvodni sistemi, prometni sistem, …). Na voljo je več različnih simulacijskih orodij za diskretno dogodkovno simulacijo: SimEvents, Arena, AnyLogic, …

Na vajah bomo uporabljali simulacijsko orodje SimEvents, ki je del simulacijskega okolja Matlab/Simulink.

Simulink in SimEvents

Modelersko in simulacijsko orodje Simulink omogoča gradnjo simulacijskega modela in izvajanje eksperimentov. Podprte so različne vrste simulacije, tako zvezna, diskretno-časovna kot diskretno-dogodkovna simulacija. Za izvedbo slednje potrebujemo SimEvents, ki je dodatna knjižnica orodja Simulink. Uporabljen je t. i. pogled na diskretno-dogodkovno simulacijo skozi sovplivanje procesov (Process Interaction world view), kjer so v ospredju entitete, ki potujejo skozi procesno usmerjeno simulacijsko shemo. Kot rezultat simulacijskega teka orodje animira prikaz prehajanja entitet-agentov preko procesne sheme, dodamo lahko še različne prikazovalnike statistik in tudi naprednejšo animacijo.

V nadaljevanju najprej razložimo osnovne gradnike diskretno-dogodkovne simulacijske sheme na preprostem primeru, ki še ni del nalog te laboratorijske vaje (primer, ki ga morate izdelati v okviru vaje, je podan nekoliko naprej v razdelku **Primer za vajo**).

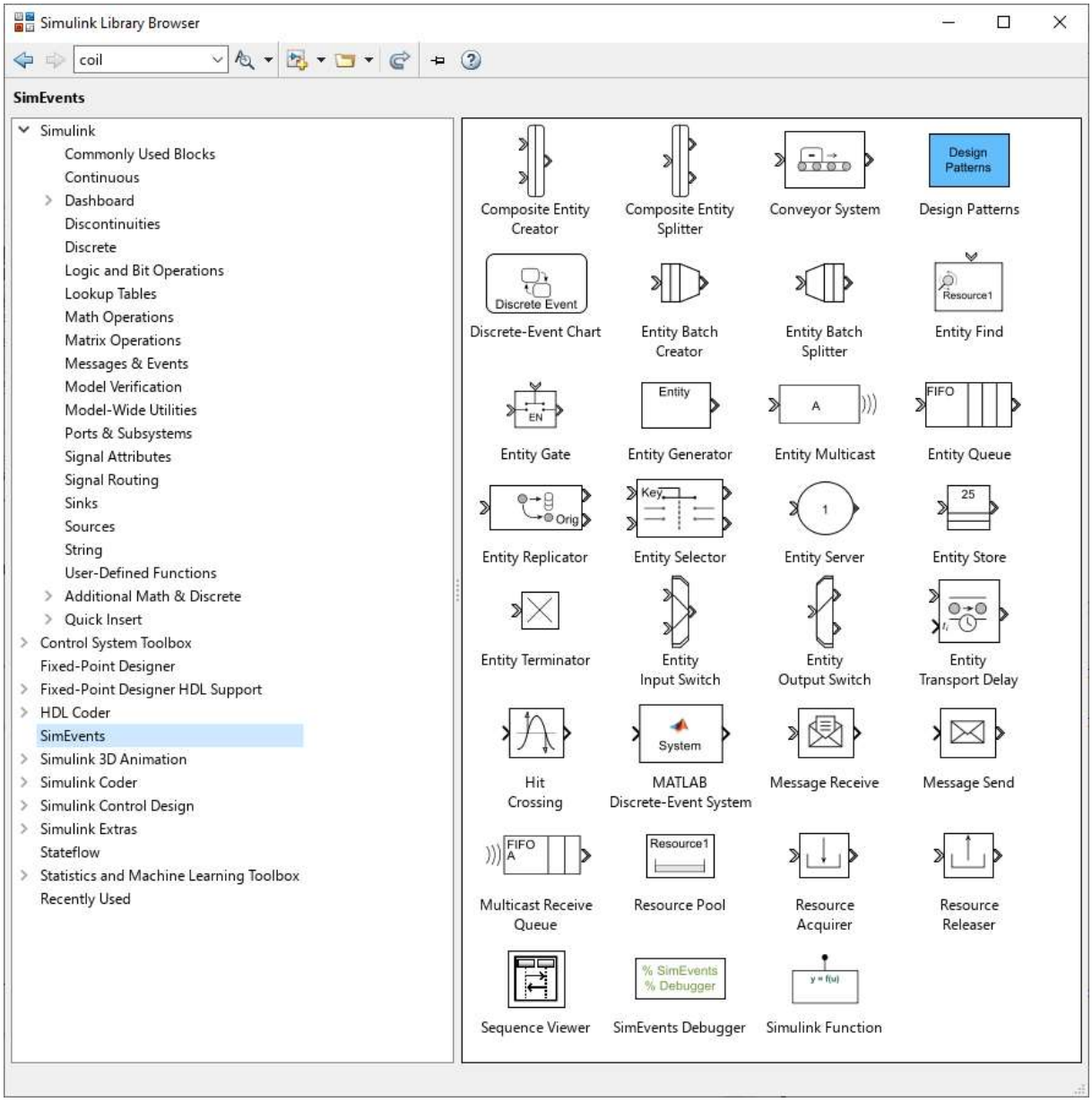
Ko zaženemo Simulink, in če izberemo opcijo Blank Model (Create Model), se odpre delovno okno z imenom untitled. Priporočljivo ga je takoj shraniti (Save), pri čemer določimo mapo za shranjevanje in ime simulacijskega modela.

Do knjižnic simulacijskih blokov dostopamo preko ukaza Library Browser, ki odpre instaliran nabor knjižnic. Med njimi je SimEvents in s klikom na ime te knjižnice odpremo okno na sliki 1.

Knjižnica SimEvents

Diskretno-dogodkovni simulacijski model gradimo v obliki procesno usmerjene simulacijske sheme in knjižnica SimEvents vsebuje nabor blokov, ki jih pri tem lahko uporabimo. Poleg teh lahko v shemi uporabljamo še nekatere standardne bloke iz knjižnice Simulink.

Kot pri drugih vrstah simulacijskih shem tudi diskretno-dogodkovno simulacijsko shemo gradimo tako, da vlečemo (ali kopiramo) bloke iz okna knjižnice SimEvents v delovno okno simulacijskega modela. Bloke povezujemo tako, da približamo kurzor izhodu/vhodu bloka in ko spremeni obliko, s pritiskom na levi gumb miške potegnemo povezavo do vhoda/izhoda drugega bloka. Pri tem lahko opazimo, da imajo bloki knjižnice SimEvents različno prikazane vhode in izhode. Votle puščice ( ) označujejo entitetne vhode in izhode, s katerimi



Slika 1: Knjižnica SimEvents

določamo pot entitet v shemi, običajne puščice pa predstavljajo signalne vhode in izhode, ki imajo enako vlogo kot pri drugih blokih v Simulinku. Entitetne povezave se izrišejo kot votle črte, signalne povezave kot običajne črte.

Kratek opis nekaterih najpogosteje uporabljanih blokov:

Entity Generator: generira entiteto ob vsakem proženju. V zavihku Entity generation konfiguracijskega okna nastavljamo način proženja Time-based/Entity-based in pri načinu Time-based še način določanja časa proženja (Time source), ki je lahko Dialog (periodo proženja nastavimo z dodatnim parametrom), Signal port (čas med dvema zaporednima generiranima entitetama določa zunanji signal) ali MATLAB action.

V slednjem primeru neposredno v konfiguracijsko okno bloka vpišemo ukaz ali zaporedje ukazov v Matlabovem skriptnem jeziku, ki izračuna spremenljivko dt; ta določa čas med zaporedno generiranima entitetama. Privzeto je vpisan ukaz dt = rand(1,1); kar pomeni naključno izbiro časa med 0 in 1 in enakomerno porazdelitev

časov v tem intervalu. Če želimo generirati entitete s drugačno porazdelitvijo časov med zaporednimi prihodi v proces, moramo ta ukaz ustrezno spremeniti. Primere najdemo v sistemu pomoči orodja (klik na Help v konfiguracijskem oknu), na voljo pa je tudi gumb Insert pattern …, ki samodejno ustvari zaporedja ukazov za najpogosteje uporabne situacije[1].

Na voljo so še drugi zavihki, kjer nastavljamo tip entitete in morebitne entiteti pridružene informacije (atribute), akcije (zaporedja ukazov), ki se izvedejo ob generiranju ali izhodu entitete iz bloka ter statistike, to so dodatni izhodni signali blokov, o katerih bo več povedano v nadaljevanju.

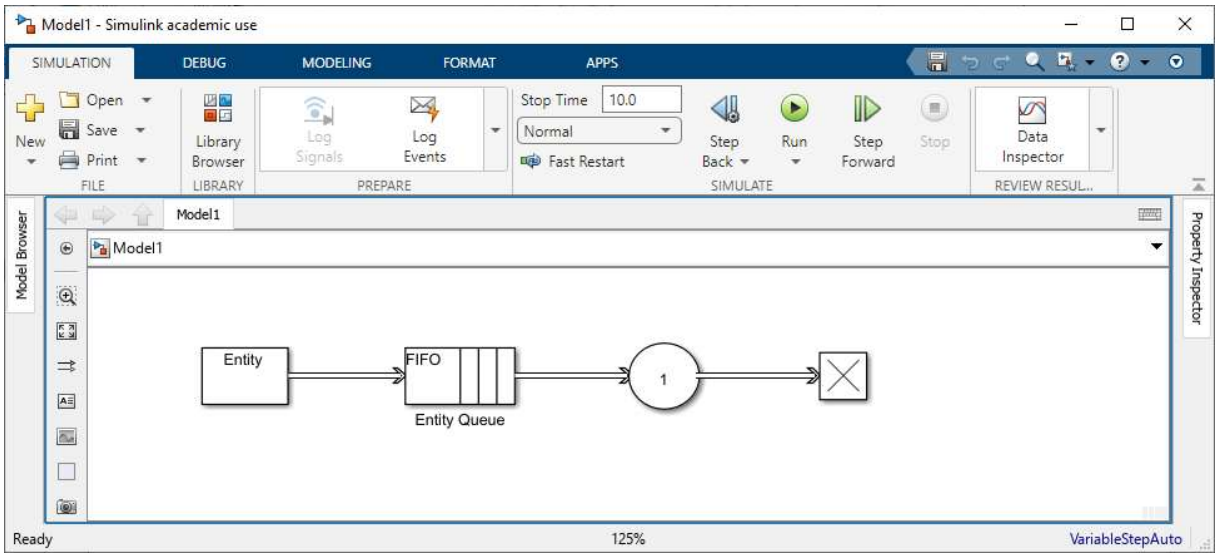
Entity Queue: modelira čakalno vrsto, kjer entitete čakajo na sprostitev vira, na katerem bodo obdelane oz. se bo izvedla ustrezna storitev. Vrsti nastavljamo kapaciteto, način prehajanja entitet (FIFO, LIFO, …), vir entitet ter druge parametre, vključno s proženjem akcij in statistikami.

Entity Server: modelira obdelavo entitet na strežniku. Tu določamo kapaciteto strežnika, porazdelitev časa storitve (podobno kot pri bloku Entity Generator) ter druge nastavitve, vključno s proženjem akcij in statistikami.

Pri določanju časa storitve (strežbe) je na voljo dodatna izbira Attribute, ki pomeni, da je informacija o času pripeta vsaki posamezni entiteti, ki pride na strežnik.

Enity Terminator je končna točka diagrama in zavrže vstopajoče entitete. Tudi tu lahko določimo akcijo, ki se sproži ob prihodu entitete, ter vključimo statistiko, ki v tem primeru podaja zgolj število prispelih entitet.

Slika 2 prikazuje značilno povezavo najpogosteje uporabljanih blokov.



Slika 2: Raba osnovnih blokov knjižnice SimEvents

[1] za več možnosti pri generiranju naključnih števil je priporočljiva namestitev orodja Statistics and Machine Learning Toolbox

Namesto kombinacije čakalne vrste in strežnika s končno kapaciteto lahko uporabimo tudi bloka **Resource Acquirer** in **Resource Releaser** v kombinaciji s strežnikom z neskončno kapaciteto. Ta kombinacija je bolj splošna in omogoča sočasno rabo različnih virov, dodati pa moramo še blok **Resource Pool** ter določiti število virov, in na katero vrsto virov se blok nanaša. Entitete nato čakajo v bloku **Resource Acquirer**, zmožnost strežnika za izvajanje storitve pa je v danem trenutku pogojena z razpoložljivimi viri.

Diskretno-dogodkovna simulacija z orodjem SimEvents

Da uporabnik zgradi simulacijski model in izvede simulacijo z orodjem SimEvents, mora izvesti naslednje korake:

1. *Gradnja osnovnega modela*. V delovnem oknu simulacijskega modela zgradimo procesni diagram, tako da vanj vstavimo in med sabo ustrezno povežemo vse potrebne bloke.
2. *Nastavljanje parametrov modela*. V model vnesemo aktualne podatke (npr. čas obdelave, potrebe po virih,…). To naredimo z klikom na simbol bloka, nakar lahko v konfiguracijskem oknu vnesemo potrebne parametre.
3. *Izvedba simulacijskega teka* modela. Ko model sestavimo, ga lahko zaženemo: izberemo ukaz Run v orodni vrstici na vrhu delovnega okna (izbran mora biti zavihek SIMULATION). V statusni vrstici ob spodnjem robu okna vidimo, da se model prevede, inicializira in zažene, rezultatov simulacije pa ne vidimo, dokler ne konfiguriramo dodatnih možnosti prikaza.

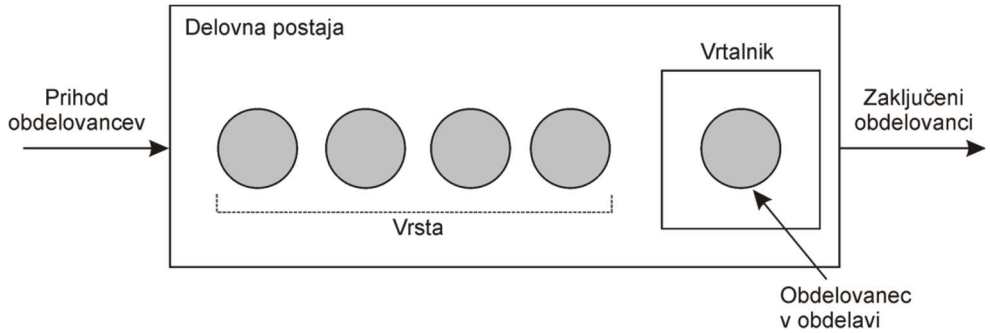
   Prva možnost prikaza je vgrajena animacija, ki jo aktiviramo z desnim klikom v delovnem oknu in izbiro Animation Speed - Fast/Medium/Slow. Če poženemo simulacijo ob vključeni animaciji, se ob vsakem prehodu entitete modro obarva pripadajoča povezava ter izvorni in ponorni blok entitete. Takšna animacija je koristna pri začetnem preverjanju delovanja simulacijske sheme, kasneje jo običajno izključimo (izbira Animation Speed - None).

   Druga možnost je opazovanje značilnih kazalnikov v okviru analize simulacijskih rezultatov, tretja možnost prikaza pa je uporabniško programirana animacija, ki jo bomo obravnavali v zadnjem delu vaje.
4. *Analiza simulacijskih rezultatov*. V SimEvents je potrebno za analizo rezultatov simulacije nastaviti beleženje določenih kazalnikov, ki se imenujejo statistike (Statistics). Kazalniki se nastavljajo na nivoju posameznih blokov, ki imajo v ta namen v konfiguracijskem oknu poseben zavihek Statistics. Zavihek odpre seznam razpoložljivih statistik, v katerem odkljukamo želene kazalnike. Značilne statistike so število entitet, ki je prešlo element, povprečna dolžina čakalne vrste, povprečen čas čakanja ipd. Če izberemo vsaj eno statistiko, se nad blokom pojavi eden ali več dodatnih izhodov z oznakami izbranih statistik. Nanje lahko priključimo standardne ponorne bloke Simulinka (Simulink/Sinks), največkrat osciloskop (Scope) in ali prikazovalnik (Display) in nato po zagonu simulacije opazujemo časovni potek izbrane statistike. Uporabnik lahko analizo tudi razširi s konfiguriranjem osciloskopov za shranjevanje v delovni prostor (Log data to workspace) ali z dodajanjem blokov za zbiranje podatkov (To Workspace) in nato izvajanjem poljubnih preračunov v Matlabu.
5. *Spreminjanje in izboljšanje modela* v skladu s potrebami uporabnika.

Primer za vajo: Vrtanje obdelovancev na delovni postaji (M/G/1)

Poglejmo si enostavni primer, ko se na delovni postaji izvaja vrtanje obdelovancev. Problem predstavimo kot sistem s čakalno vrsto z enim strežnikom (M/G/1).

Neobdelani obdelovanci prihajajo na delovno postajo, kjer se jim z vrtalnikom zvrta luknja, nato pa delovno postajo zapustijo (slika 3). Ko pride obdelovanec na delovno postajo, se takoj postavi na mizo za vrtanje. V primeru, ko je vrtalnik zaseden s še prejšnjim obdelovancem, pa mora prihajajoči obdelovanec počakati v vrsti, kjer velja pravilo FIFO (*First-In First-Out*). To prestavlja logično strukturo modela.



Slika 3: Primer delovne postaje

Časovno skalo pri opazovanju delovanja sistema bomo definirali v minutah. Predpostavimo, da so časi med zaporednimi prihodi obdelovancev porazdeljeni eksponentno z matematičnim upanjem 3 min, čas vrtanja pa sledi trikotni porazdelitvi med 1 in 4 min z najverjetnejšim trajanjem 1,5 min. Predpostavimo tudi, da ni omejitev glede števila čakajočih obdelovancev.

Pri tem nas zanima več kazalnikov, ki govorijo o učinkovitosti delovne postaje (*Output performance measures*). Nekateri od teh kazalnikov so:

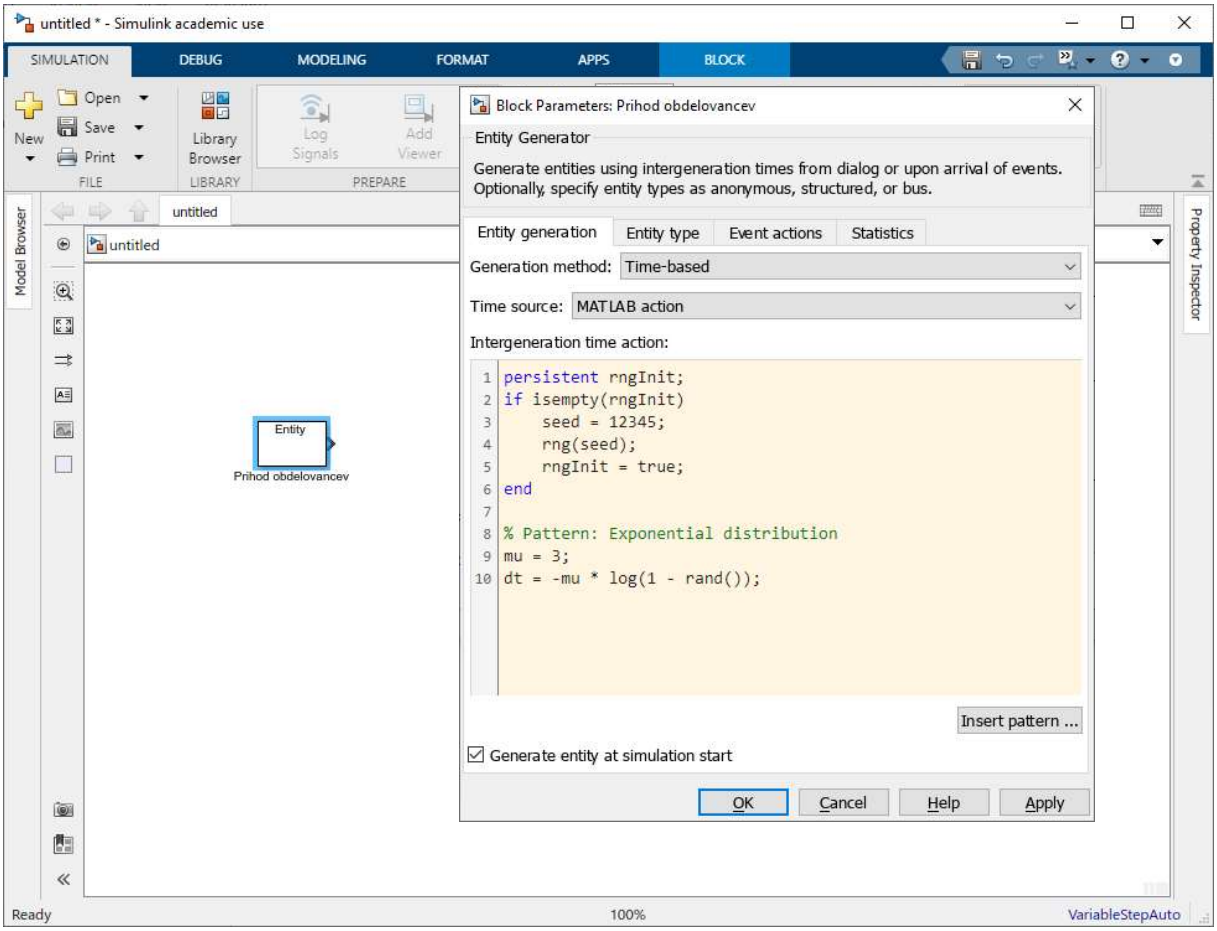
- Skupno proizvedeno,
- Povprečni in maksimalni čas čakanja v vrsti,
- Povprečno in maksimalno število obdelovancev v vrsti,
- Povprečni in maksimalni čas, ko je bil obdelovanec v obdelavi,
- Izraba proizvodnega vira ρ (*Utilization*),
- Povprečno število obdelovancev na delovni postaji,
- Povprečni pretočni čas …

Objekti, ki gredo skozi proces, so v našem primeru obdelovanci in v orodju SimEvents le-te predstavimo z entitetami.

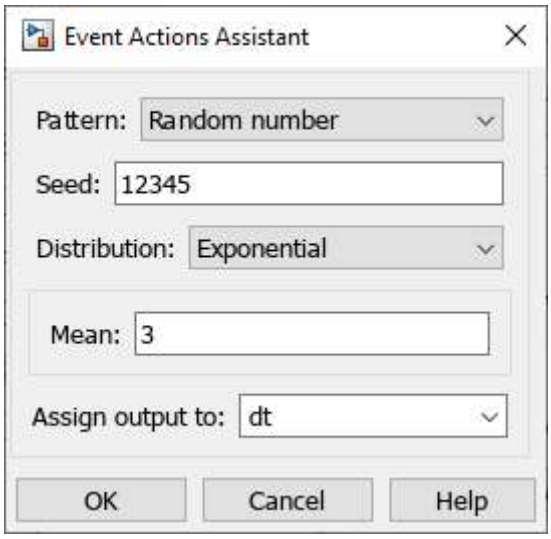
Nov model ustvarimo preko ukaza *New/Simulink Model* v Matlabu, ali pa najprej poženemo Simulink in nato izberemo Blank Model. V prvem koraku modeliranja je potrebno določiti procesni diagram poteka. V orodni vrstici novega okna kliknemo Library Browser, v navigacijskem drevesu okna s knjižnicami izberemo SimEvents ter povlečemo potrebne gradnike v delovno okno modela. Najprej dodamo blok *Entity Generator* (slika 4). Z njim predstavimo vir prihajanja novih obdelovancev. Blok poimenujemo »Prihod obdelovancev«.

Ob dvokliku na blok se odpre konfiguracijsko okno, v katerem nastavimo zakonitosti prihajanja obdelovancev. Obdelovanci prihajajo naključno in čas med prihodoma dveh obdelovancev (*Interarrival time*) sledi obnašanju, ki ga lahko predstavimo z eksponentno porazdelitveno funkcijo s srednjo vrednostjo 3 minute (slika 4). To ustreza stopnji prihoda

0,33 na minuto. Kodo v razdelku Intergenerate time action: lahko vpišemo ročno, lahko pa si pomagamo z generatorjem kode, ki ga aktiviramo preko gumba Insert pattern …. Nato izberemo *Random number* in v novem oknu nastavimo parametre, kot prikazuje slika 5. Nastavitve potrdimo z OK.



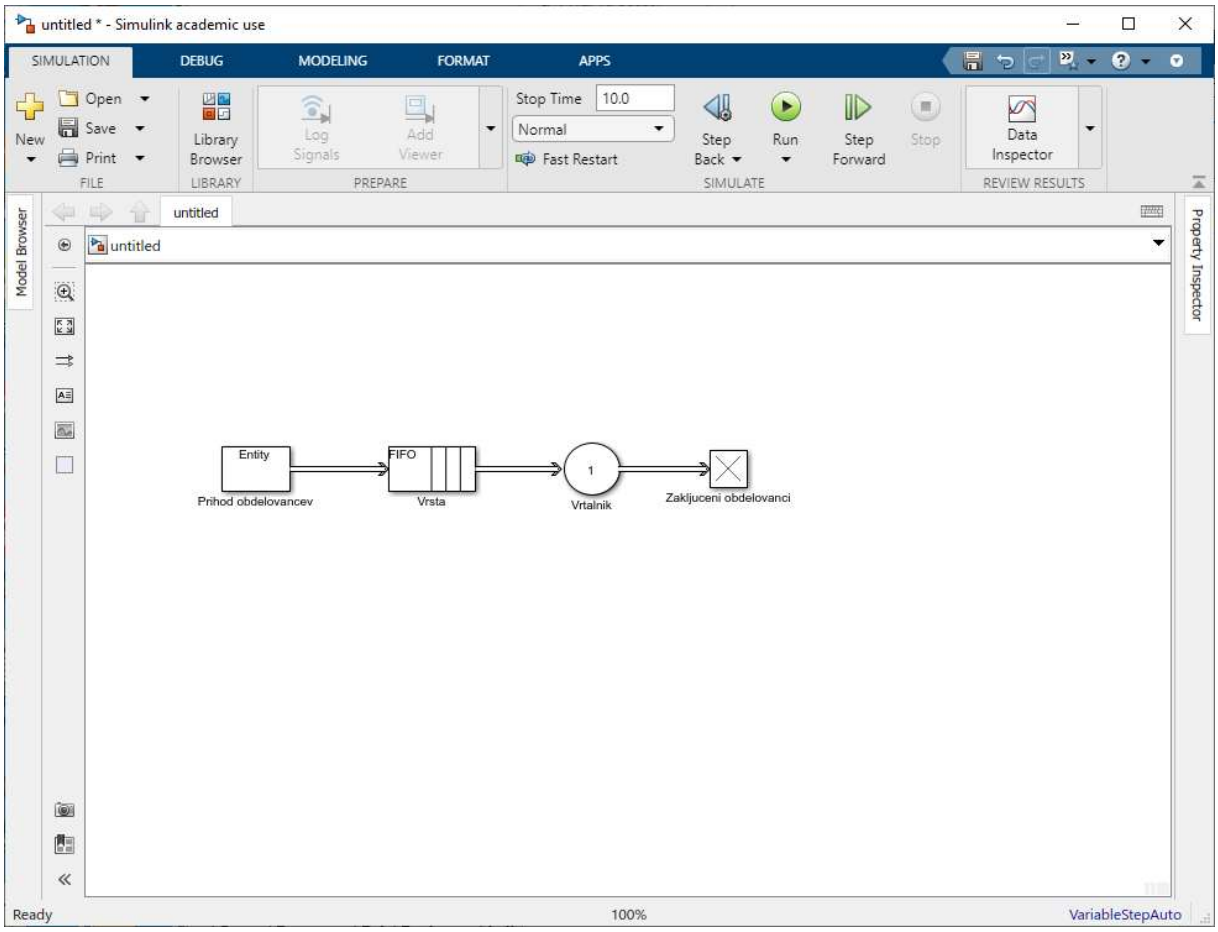
Slika 4: Blok Entity Generator – prihajanje obdelovancev



Slika 5: Generiranje kode za eksponentno porazdelitev časov med prihodi entitet

Tip entitet, ki jih modul generira, za zdaj pustimo na prednastavljeni vrednosti »Structured«, pav tako ne spreminjamo entitetam pridruženih atributov (to bomo spremenili kasneje v drugem delu vaje, ko bomo zbirali dodatne statistike).

Modelu dodamo še bloke *Entity Queue*, *Entity Server* in *Entity Terminator*, s katerimi predstavimo tok obdelovancev skozi proces, in jih povežemo (slika 6).

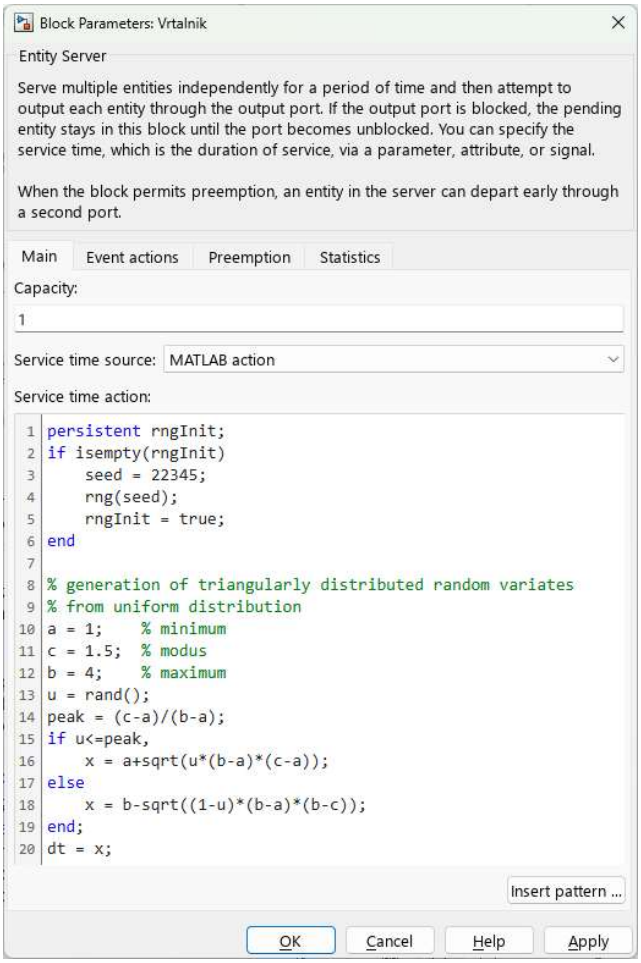


Slika 6: Procesni diagram poteka simulacijskega modela delovne postaje

Sledi nastavljanje parametrov. Bloku *Entity Queue* spremenimo kapaciteto (Capacity: inf) v zavihku Statistics pa izberemo kazalnike, ki jih potrebujemo za analizo rezultatov: Number of entities in block, Average wait, Average queue length.

Da bomo bloku *Entity Service* lahko nastavili trikotno porazdelitev strežnih časov, moramo podobno kot pri generiranju entitet nastaviti Service time source: MATLAB action, nato pa moramo ustrezno kodo vpisati ročno, SimEvents namreč ne vsebuje generatorja kode za trikotno porazdelitev. Primer ustrezne kode prikazuje slika 7. Pomembno je, da začetno stanje generatorja naključnih števil (seed) nastavimo drugače kot pri generiranju entitet, da bomo dobili neodvisna naključna procesa. Kapaciteto strežnika pustimo na prednastavljeni vrednosti 1, dodatno pa nastavimo še zbiranje statistik, in sicer: Average wait in Utilization.

V nadaljevanju je potrebno v bloku Entity Terminator vključiti še statistiko, ki bo omogočala beleženje skupnega števila obdelanih obdelovancev, to je v danem primeru skupno število entitet, ki prispejo do konca poti skozi proces: Number of entities arrived.



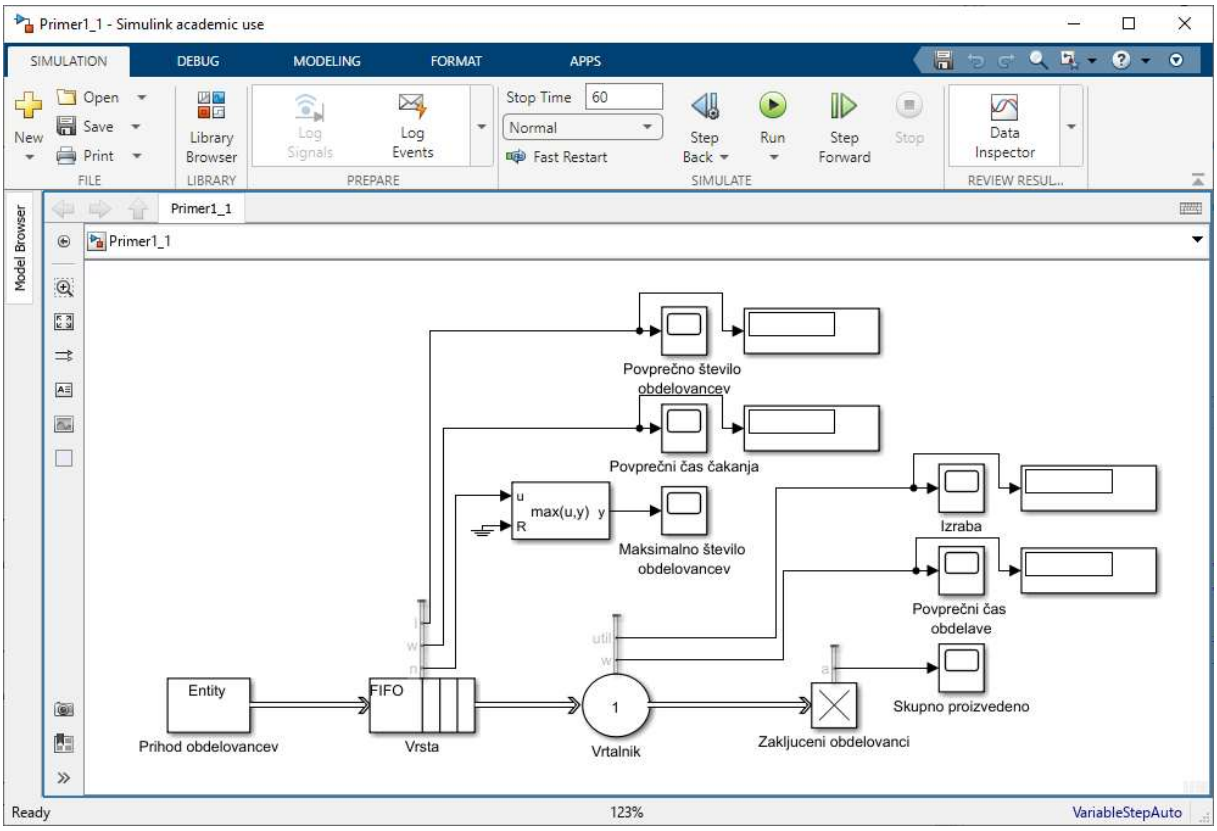
Slika 7: Parametri vrtalnika na delovni postaji

V zadnjem koraku izgradnje modela je potrebno na vse dodatne izhode, ki so se pojavili ob vklopu statistik pri posameznih blokih, priključiti prikazovalnike. Izjema je določanje maksimalnega število obdelovancev v vrsti, ki ga dobimo s preračunom statistike n pri čakalni vrsti. Ta izhod zato priključimo na blok MinMax Running Resettable (knjižnica Math Operations), izhod tega bloka pa na prikazovalnik. Za prikazovalnike je najbolje uporabiti kar blok Scope iz Simulinkove osnovne knjižnice Sinks. Priporočljivo je vsak blok poimenovati z imenom prikazane statistike.

Nastavitve izvajanja simulacijskih tekov uporabnik določi preko orodne vrstice v zgornjem delu delovnega okna modela. Za začetni test nastavimo, da naj en simulacijski tek traja 60 minut (Stop Time: 60), ostale nastavitve pustimo na prednastavljenih vrednostih.

Model je na tej točki priporočljivo shraniti, in če ga še nismo, ga moramo tudi poimenovati. Primer do opisane faze dokončanega modela z imenom Primer1 prikazuje slika 8.

Simulacijo modela zaženemo z ukazom *Run* v orodni vrstici ali s kombinacijo tipl Ctrl+T. V statusni vrstici okna se najprej izpiše obvestilo o prevajanju modela, nato o inicializaciji in teku modela. Če v delovni površini kliknemo desni gumb, izberemo Animation Speed različen od None in ponovno poženemo simulacijo, se izvajanje upočasni, ob vsakem prehodu entitete pa se obarva del sheme, kjer se prehod odvija.



Slika 8: Simulacijski model delovne postaje

Če odpremo prikazovalnike, lahko opazujemo tudi časovni potek statistik. Pri opisanih nastavitvah lahko razberemo, da je po koncu simulacije povprečno število obdelovancev v vrsti približno 0,72, povprečni čas čakanja približno 2,4 minute, maksimalno število obdelovancev v vrsti 3, izraba strežnika približno 0,7, povprečni čas obdelave približno 2,6 in skupaj proizvedeno 15 izdelkov. Te vrednosti so lahko tudi drugačne, če drugače nastavimo parameter seed generatorjev naključnih števil.

Za nadaljnje eksperimente je smiselno izključiti animacijo prehajanja entitet, ki močno upočasni izvajanje simulacije.

Naloge (1. del):

1. Izdelajte simulacijski primer v skladu z zgornjimi navodili in preizkusite njegovo delovanje.
2. Preizkusite in komentirajte, kaj se zgodi, če povečamo čas med prihodoma dveh obdelovancev. In v primeru zmanjšanja časovnega intervala med prihodi?
3. Preizkusite in komentirajte kaj se zgodi, če povečamo/zmanjšamo povprečni čas intervala procesiranja (vrtanja).
4. Vrnite parametre modela nazaj **na izhodiščne vrednosti** in preverite ujemanje zbranih statistik s teoretično napovedanimi vrednostmi (enačbe za sistem M/G/1). Kateri rezultat se ujema in kateri ne? Kaj bi morali spremeniti, da bi se vsi rezultati (približno) ujemali s teoretično napovedanimi vrednostmi? Ustrezno popravite simulacijske nastavitve in ponovno preverite ujemanje. V poročilo vključite **tabelo s primerjavo** simulacijsko določenih statistik in teoretičnih napovedi.

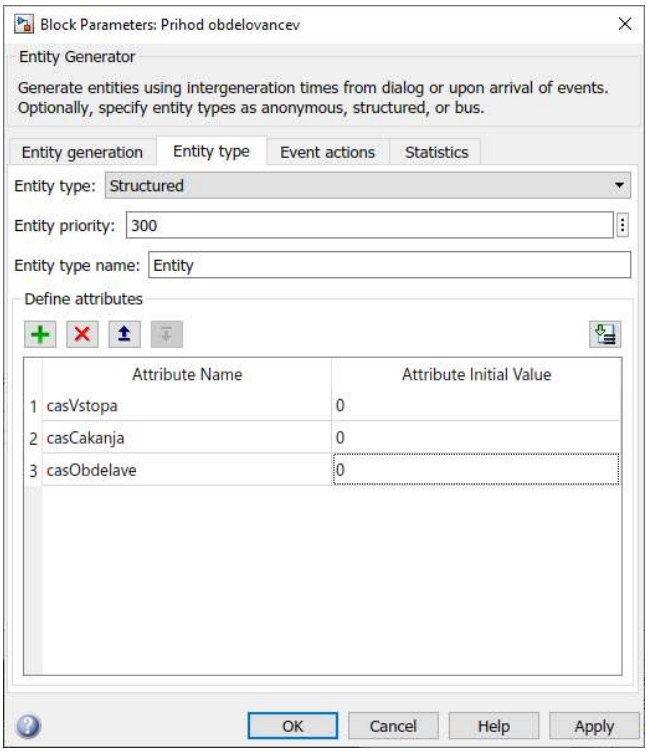
V nadaljevanju si bomo ogledali še naprednejše zbiranje in analizo statističnih parametrov ter dodatne možnosti animacije simulacijskega modela v orodju SimEvents.

Dodatne statistike

Od v opisu naloge zahtevanih kazalnikov smo do tu večino že realizirali, manjkata pa kazalnika povprečnega števila obdelovancev na postaji in povprečnega pretočnega časa, kar lahko izračunamo iz obstoječih kazalnikov. Poleg tega manjka še izračun kazalnikov maksimalnega časa čakanja v vrsti in maksimalnega časa obdelave, ki ju ne moremo neposredno realizirati preko nastavitev blokov.

Za izvedbo računanja obeh maksimalnih časov bomo uporabili možnost prenašanja informacij z entitetami preko t. i. atributov.

Vsaki entiteti dodamo informacijo, ob katerem času je bila generirana, kar bo nato omogočilo spremljanje časa potovanja entitete po procesni shemi. V ta namen dopolnimo nastavitev generatorja entitet, tako da definiramo tri atribute, kot kaže slika 9.



Slika 9: Atributi entitete za spremljanje časov

Dodatno moramo definirati tri Simulinkove funkcije (blok Simulink Function iz knjižnice User-Defined Functions). V prvi izbrišemo vhodni parameter (), namesto njega pa dodamo blok Digital Clock, ki mu nastavimo Sample time -1, in ga povežemo na izhod funkcije. V primeru smo Izhod poimenovali t, samo funkcijo pa simClock. Nato pri bloku čakalne vrste (Vrsta) definiramo vstopno akcijo, tako da izberemo zavihek Event actions, kliknemo Entry in vpišemo v okno pod Entry action:

```
entity.casVstopa=simClock;
```

S tem dosežemo, da se bo ob vstopu entitete v vrsto trenutni čas zapisal v atribut casVstopa.

Drugo in tretjo funkcijo bomo uporabili za branje atributov. Potrebno jima je spremeniti ime, npr. v observeAttrib1 in observeAttrib2, izhodni parameter funkcije pa pri obeh nadomestimo s Simulinkovim izhodom iz bloka (Out1 iz knjižnice Sinks). S tem se na bloku funkcije pojavi priključek za izhodni signal.

Preostane še, da v shemi definiramo akcije, ob katerih se bodo računali časi. Ena od možnosti je, da vse izračune vežemo na strežnik. Čas čakanja v vrsti izračunamo ob vstopu entitete na strežnik, tako da vstopno akcijo Entry definiramo kot:

```
entity.casCakanja=simClock()-entity.casVstopa;

observeAttrib1(entity.casCakanja);
```

S tem smo izračunali čas čakanja in ga prenesli na izhod funkcije observeAttrib1.

Podobno izračunamo čas obdelave ob zaključku strežbe z akcijo Service complete, npr.:

```
entity.casObdelave=simClock()-entity.casVstopa-entity.casCakanja;

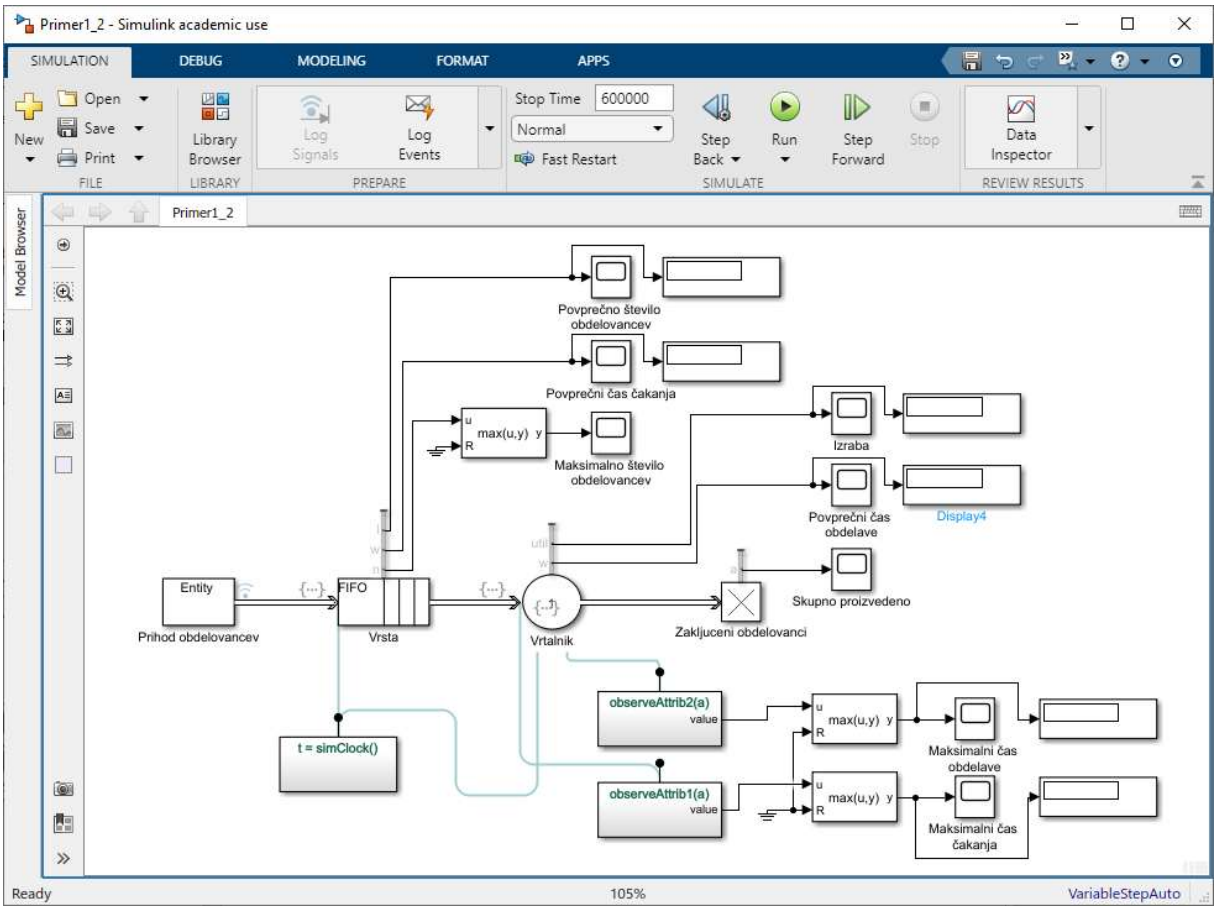
observeAttrib2(entity.casObdelave);
```

Ta čas se pojavi kot signal na izhodu funkcije observeAttrib2.

Preostane še, da podobno kot pri dolžini vrste signala povežemo na blok MinMax Running Resettable in nato na ustrezen prikazovalnik.

Primer na opisan način dopolnjene sheme prikazuje slika 10. Na sliki poleg dodanih funkcij vidimo tudi, iz katerih blokov so klicane. Če želimo videti te povezave, v zavihku DEBUG izberemo Information Overlays in kliknemo na Function Connectors.

Poleg tega se pri blokih z definiranimi akcijami pojavi tudi oznaka {…}, ki nakazuje, pri katerih dogodkih so definirane akcije (oznaka se pojavi na vhodu, izhodu ali znotraj bloka).



Slika 10: Model z dodatnimi statistikami

Ko poženemo simulacijo, lahko iz prikazovalnikov razberemo oba maksimalna časa.

Na podoben način lahko ustvarimo še različne druge kazalnike, ki jih potem znotraj sheme še poljubno obdelamo, lahko pa jih tudi prenesemo v Matlab za nadaljnjo obdelavo.

Zadnja manjkajoča kazalnika dobimo tako, da seštejemo povprečje obdelovancev v vrsti in povprečje obdelovancev v vrtalniku (izkoriščenost vrtalnika) – na ta način dobimo povprečno število obdelovancev na postaji; ter da seštejemo povprečni čas čakanja in povprečni čas obdelave – dobimo povprečni pretočni čas.

2D animacija virov in vrst

V prvem delu smo že spoznali preprosto vgrajeno animacijo, ki signalizira prehajanje entitet. Med simulacijskim tekom pa lahko vire in čakalne vrste ponazorimo tudi bolj nazorno, z 2D ali 3D animacijo. Na tem mestu bomo modelu, ki smo ga zgradili v prvem delu, dodali 2D animacijo.

Animacijske učinke dosežemo z objektom razreda simevents.SimulationObserver, ki ga moramo ustvariti za konkreten simulacijski model. Ob inicializaciji se ustvarjeni objekt poveže s simulacijskim modelom. Po vsakem zagonu simulacije se nato kličejo metode objekta ob določenih dogodkih, kar omogoča izvedbo različnih operacij, med drugim tudi izris grafičnega prikaza in sprotno spreminjanje lastnosti grafičnih objektov glede na stanje simulacije.

Za lažje delo je primeren objekt že ustvarjen in ga lahko prenesete iz spletne učilnice. Za pravilno delovanje je potrebno prilagoditi imeni blokov za čakalno vrsto in strežnik v funkciji getBlocksToNotify – imeni se morata ujemati z dejanskimi imeni v izdelani simulacijski shemi.

Objekt nato inicializiramo z ukazom: obj = queueingSystemAnimator('ime modela'), pri čemer navedemo ime modela v odprtem oknu (npr. obj = queueingSystemAnimator('Primer1_1')).

Po zagonu simulacije se izriše slika, v kateri se sproti izrisuje položaj entitet, sproti pa se izpisujejo tudi nekatere statistike. Pri tem je smiselno upočasniti izvajanje simulacije in skrajšati maksimalni čas simulacijskega teka.

Naloge (2. del):

1. Dopolnite simulacijski model z izračunom dodatnih kazalnikov na osnovi atributov, kot je opisano zgoraj.
2. Dodajte računanje povprečnega števila obdelovancev na postaji in računanje povprečnega pretočnega časa. Preverite ujemanje s teoretično napovedjo vrednosti kazalnikov. Prikažite rezultate v tabeli.
3. Konfigurirajte model kot sistem M/M/1 z enako stopnjo strežbe (uporabite enako matematično upanje porazdelitve časov vrtanja) in za zbrane statistike preverite ujemanje vrednosti, ki izhajajo iz teoretičnih relacij, z vrednostmi, ki jih dobite s simulacijo. Rezultate prikažite v tabeli.
4. Preverite veljavnost Littlovega zakona za čakalno vrsto in za celoten sistem: s pomočjo simulacije določite povprečno število obdelovancev v vrsti oz. v sistemu ter povprečni čas čakanja v vrsti oz. povprečni pretočni čas sistema, nato vstavite izmerjene vrednosti v enačbo in preverite ujemanje leve in desne strani enačbe.
5. Preverite, ali se veljavnost Littlovega zakona ohrani, če kot porazdelitev časov vrtanja uporabimo trikotno porazdelitev.
6. Izdelajte preprosto 2D animacijo čakalne vrste in stroja, kot je opisano zgoraj. Preizkusite delovanje animacije.

7. **Dodatna naloga**: Dodajte še eno postajo z vrtalnikom in vstavite razcep poti entitet (blok Entity Output Switch), tako da bodo v razmerju verjetnosti 1:1 potovale na prvotno in novo delovno postajo. Dodajte zbiranje statistik za novo postajo in preverite veljavnost Littlovega zakona v tem primeru.